

Ships that pass in the night: Competition in the fossil record (Coded in Julia)

Jeet Sukumaran and Maritza Sardina

Table of contents

1 Part 1: Getting to know PaleobiologyDB (Using Navigator and Julia)	2
2 Part 2: Exploring your surroundings	7
3 Part 3: Examining patterns through geological time (by creating diversity curves)	10
4 References (for Original Activity)	16

The original “Ships that pass in the night: Competition in the Fossil Record” was made by John Fronimos from Vassar College and Philip Novack-Gottshall from Benedictine University. It was originally published on May 2nd, 2018, and last revised on March 2nd, 2023. This tutorial can be accessed from <https://serc.carleton.edu/teachearth/activities/204123.html> or on the “Resources” tab of PaleobiologyDB under the “Lessons Plans and Educational Activities” section. This tutorial incorporates components of the original tutorial using a mix of both Julia code and written questions, as well as use of the PaleobiologyDB’s Navigator tool as a reference.

Major events in paleontological history have resulted in changes in the biodiversity of numerous species, genera, and major groups of organisms. Many fossils belonging to these groups can be found using PaleobiologyDB’s Navigator tool through fossil sites scattered across the world. This is thanks to the vast availability of resources like Navigator and the Internet, which allow us to access records of these fossil occurrences almost immediately. One such goal of this research is to detect and configure ways to reduce or resolve problems surrounding biodiversity changes in our modern world (Fronimos and Novack-Gottshall, 2018).

Hence, the goal of this tutorial is to grow familiarity with both Navigator and the Julia environment (for your coding sections), and to test hypotheses about these major events and changes in diversity using these resources. This includes creating a diversity curve that pertains to diversity and interval data regarding three marine groups of organisms on the sea floor.

1 Part 1: Getting to know PaleobiologyDB (Using Navigator and Julia)

Before using the PaleobiologyDB Julia interface, you should become familiar with PaleobiologyDB (<https://paleobiodb.org/#/>) and its tools, including the Navigator. Navigator is a map-based tool that can be used to access, visualize, and retrieve data from the PBDB. Navigator consists of several working components, including a search field, a world map with “Dots” representing points in the geological time, a toolbox that enables you to alter the map to your interests, and a geological time scale at the bottom of the webpage. By going on <https://paleobiodb.org/navigator/>, you can notice several of these components throughout the webpage, as shown below:

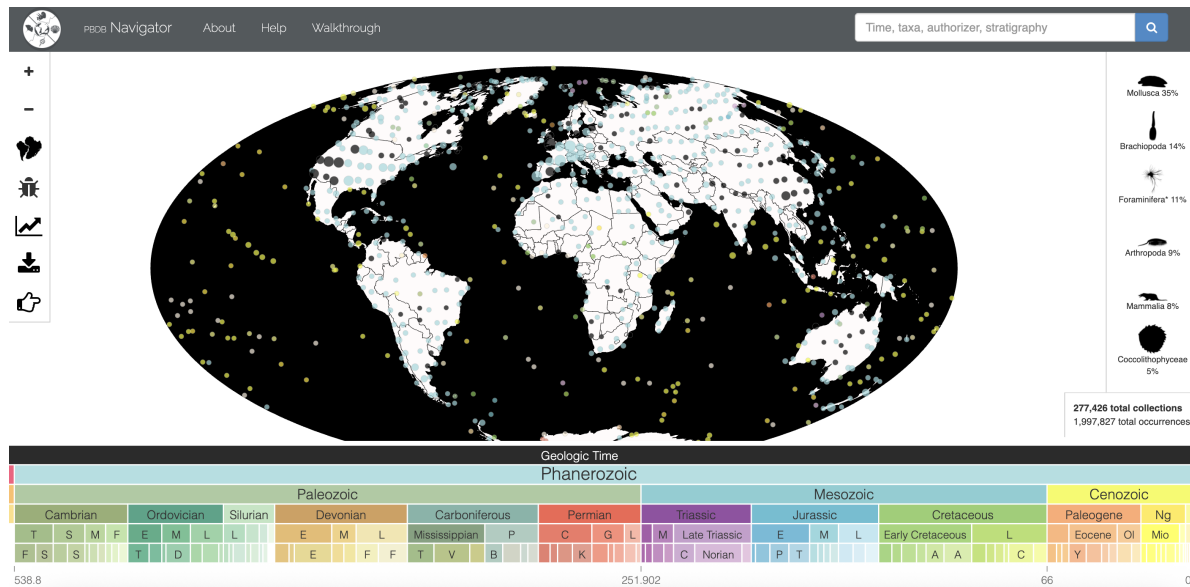


Figure 1: Screenshot of the Navigator tool webpage, which depicts the world map with circles

For now, you will take your time in getting familiar with Navigator and its components, including the world map, the search field, and the toolbox, which are essential for parts of this tutorial. The first few questions (1a to 2b) of the activity are on the official activity file (from Fronimos & Novack-Gottshall, 2018), and they should be followed throughout this specific section of the tutorial. As such, you should have the PDF for the original activity open during the introduction/basics of Navigator section (in **Part 1: Getting to Know the Paleobiology Database (PBDB)**).

NOTE: For the portion below marked by “start” and “end”, and for other portions pasted from the activity, you should follow along the original activity under Fronimos and Novack-Gottshall (2018). As such, most of the questions from the original activity will be cited in-text.

start

Question 1a: What do the circles on the map represent? [Fronimos & Novack-Gottshall, 2018]

Question 1b: What are some of the circles larger than others? [Fronimos & Novack-Gottshall, 2018]

Question 1c: What do the colors on the map represent? [Fronimos & Novack-Gottshall, 2018]

Question 2a: What kind of information can you get from the “General” tab on Navigator? [Fronimos & Novack-Gottshall, 2018]

end

Now, using Navigator, find your birthplace on the world map and get as close to it as possible. Click on the circle nearest to your location and look up all fossil collections within the circle. If there are multiple collections embedded in the circle, then select the first one. For this part of the tutorial, we’ll be using both the “General” tab to know more about our fossil collections and the `pbdb_occurrences` function in the “PaleobiologyDB.jl” Julia package to answer the questions in the Activity PDF. After answering the first few questions, you will need to set up Julia to begin making our code.

First we’ll need to set up our Julia environment. This is done by loading several packages into our environment, whether specific or generalized (as such in the tutorial):

```
using PaleobiologyDB, PaleobiologyDB.Taxonomy, DataFrames
using Makie, CairoMakie, GeoMakie, ColorSchemes

PaleobiologyDB.set_autocaching!(true);
```

Auto-caching is enabled to store the results of all instances of PaleobiologyDB queries locally. This is to ensure that if repeated functions are used within your environment, the same data can be used without having to re-download it again.

For this section, we’ll use a species that was found from a single collection in San Diego, specifically the Mid-City district. Say, for example, the species that you found was *Puffinus kanakoffi*, and you want to know more about that species. In this case, we are analyzing a single datum, which wouldn’t generate the best results. Hence, it would be in your best interest to expand your research beyond your hometown and analyze multiple diverse collections from around the world, such as La Huasteca Canyon in Monterrey (see below).

Before we begin analyzing our data, we’ll use the command `pbdb_collection` to obtain the exact geospatial details about our collection and its location using this code:

```
collection_pk = pbdb_collection("col:178006", show = ["loc", "stratext"], extids = true)

show(describe(collection_pk), allrows = true)
```

A `show` command is used in conjunction with the `describe` command to list all rows and miscellaneous details about the DataFrame we've generated using `pbdb_collection`. Now, you'll need to call our data using `pbdb_occurrences`, with the species name being represented by its `base_name`. The “;” used in the code below represents the separation of a syntax argument from a keyword argument, and it might be useful for suppressing certain outputs from forming (although it isn't always used). Contrary to popular belief, putting in a limit (e.g., `limit = 1000`) might not be as useful, as it might greatly warp our data to fit certain biases in research. Thus, our code will be:

```
occs = pbdb_occurrences(
  ;
  base_name = "Puffinus kanakoffi",
  show      = "full",
  vocab     = "pbdb",
  extids    = true
)
```

There are several commands that can be used to describe and show the data for *P. kanakoffi*, such as `show`, `foreach`, `describe`, and more. `foreach` is used with two arguments (`println` and `names(occs)`) to list the columns of the entire DataFrame. `describe` is typically used to summarize the DataFrame with column names, element types, and other descriptors. ‘show’ (with the additional function of `allrows=true`) is used to list everything from the DataFrame.

As such:

```
foreach(println, names(occs))

describe(occs)

show(describe(occs), allrows=true)
```

With the `combine` and `groupby` functions, we can see how the fossil records for *P. kanakoffi* are distributed across different taxonomic ranks.

```
combine(groupby(occs, :accepted_rank), nrow)
```

Through this output, there are only 11 records within the entire collection, which greatly contrasts the collections typically seen in San Diego (our example) with other more diverse collections from around the world. Specifically, a collection of fossils (particularly in one site) is always just a singular datum, so it might help to broaden your research to entire different collections. In this example, San Diego doesn't exactly have a lot of diverse fossil collections, since many of them were documented to stem from later periods in geological history, like the Paleogene and the Neogene (hence resulting in a lack of midpoint age ranges). Hence, we will leave the data query as it is.

Now, you might want to answer this next question: "Where do you come from?" There are many places around the world with diverse collections, like Wisconsin, Morocco, and much more. Say, for example, you found roughly 18 collections within the area of Monterrey in Mexico, and all of these collections were located within La Huasteca Canyon. You'll want to look through each group of occurrences to find useful information, such as common phyla and classes (taxonomic ranks). Furthermore, these collections seem to come from the Early Cretaceous epoch, allowing for a more significant range for midpoint ages. Hence, you'll need to obtain the queries from your collection using the workflow established above.

This time, we will be assigning variable names to our main phylum seen in these collections (e.g., Mollusca, `animal = "Mollusca"`) and the specific period we want to analyze from (`period = "Late Aptian"`):

```
coll_monterrey = pbdb_collection("col:220702", show = "full", extids = true)

show(describe(coll_monterrey), allrows = true)

animal = "Mollusca"
period = "Late Aptian"

occs_mon = pbdb_occurrences(
  base_name = animal,
  interval = period,
  show = "full",
  vocab = "pbdb",
  extids = true
)

foreach(println, names(occs_mon))

describe(occs_mon)

show(describe(occs_mon), allrows = true)

combine(groupby(occs_mon, :accepted_rank), nrow)
```

After you are finished with this particular code, you can move on to these next pieces of code below.

In order to filter out the DataFrame to solely include species and sub-species occurrences, we can filter out the data using ‘filter’, which only keeps certain rows fitting that condition.

```
clean_taxonomy_flt =  
    row -> row.accepted_rank == "species" || row.accepted_rank == "subspecies"  
  
occs_mon_species = filter(clean_taxonomy_flt, occs_mon)
```

This should give you a filtered DataFrame with solely species and sub-species occurrences, which were filtered out using a custommade function within a labeled variable called `clean_taxonomy_flt`. Now, to calculate direct midpoint age (Ma) measurements for these filtered fossil occurrences, we can select these ranges using the following code:

```
occs_mon_species[:, r"._ma.*"]
```

This is an example of regular expressions in Julia. For context, a regular expression is a string of characters and symbols used to search and match certain patterns within our DataFrame. Along with a colon, “:”, to ensure that the function applies to every row, this sequence of characters (beginning with “r...”) is asking us to search for and match to every single column with the pattern “_ma” representing midpoint age. Hence, this type of expression converts strings of text into viable outputs that we can analyze from.

With this in mind, this should give you another filtered DataFrame with only the columns that include the midpoint age (Ma) measurements for filtered fossil occurrences of Mollusca. To check how many direct MA and MA error measurements are present, you can use `println` to determine which age columns for your fossil occurrences to use. “direct_ma_value” represents the highest precision, but the lowest coverage for your occurrences. “max_ma” and “min_ma” bracket the precise age range for your occurrences, which ensures broader coverage. “ma_error” represents the uncertainty for your occurrences and their age columns/measurements. In this case, we will be sticking to just “max_ma” and “min_ma”.

```
println("Max MA measurements: ", nrow(dropmissing(occs_mon_species, r"._max_ma.*")))  
println("Min MA measurements: ", nrow(dropmissing(occs_mon_species, r"._min_ma.*")))
```

In our scenario, we want to retain both “max_ma” and “min_ma” measurements for our fossil occurrences.

```
occs_mon_ages = dropmissing(occs_mon_species,
    [:max_ma, :min_ma])
```

In terms of spatial completeness, we will need modern coordinates for our fossil occurrences. First, we'll need to check the coverage for both kinds of coordinates using `println`, then we will retain both values to clean up our DataFrame with a new variable, like `occs_clean`. As such:

```
modern_coords = nrow(dropmissing(occs_mon_ages, [:lng, :lat]))
println("Records with modern coordinates: ", modern_coords)

occs_mon_clean = dropmissing(occs_mon_ages, [:lng, :lat])
```

Finally, you can use your cleaned DataFrame to print out the number of different taxonomic records present for your species, using a plethora of functions previously mentioned. These include `combine` and `groupby` (to determine the distribution of taxonomic ranks in your chosen species), and `println` (to print out the number of records available for your species and its cleaned DataFrame). This is done using the code below:

```
species_counts = combine(groupby(occs_mon_clean, :accepted_name), nrow)
genus_counts   = combine(groupby(occs_mon_clean, :genus), nrow)
family_counts  = combine(groupby(occs_mon_clean, :family), nrow)

println("Species: ", nrow(species_counts))
println("Genera:   ", nrow(genus_counts))
println("Families: ", nrow(family_counts))
```

start

Question 2b: Now that you've played around with `pbdb_occurrences` in Julia, what kind of information can you get from the fossil occurrences of your species? Make sure to write down the name of your species (in the correct formatting) and exact location. [based on Fronimos & Novack-Gottshall, 2018]

2 Part 2: Exploring your surroundings

Now, using Navigator, locate the fossil areas for both Poughkeepsie (in the south) and Saugerties (in the north), then answer these two questions using both Navigator and other information that you can find online. Questions 3a, 3b, and 4 are all lifted from the official activity file.

Question 3a: What time period do most of the fossil sites in this area come from? [Fronimos & Novack-Gottshall, 2018]

Question 3b: What types of environments were present in this area during that time period? [Fronimos & Novack-Gottshall, 2018]

end

With Navigator, locate the fossil site Onondaga Formation in Kingston, look closely at each phyla and class found within the area, and write down each name. Underline the phylum or class that has the most representation (or occurrences) in the area, then use `pbdb_occurrences` in the Julia interface to find out more about that phylum or class. In our example, we will be looking at *Mollusca*, using this workflow that was previously established in Part 1 below.

```
coll_onondaga_1 = pbdb_collection("col:129948", show = "full", extids = true)
show(describe(coll_onondaga_1), allrows = true)

coll_onondaga_2 = pbdb_collection("col:130152", show = "full", extids = true)
show(describe(coll_onondaga_2), allrows = true)

coll_onondaga_3 = pbdb_collection("col:130260", show = "full", extids = true)
show(describe(coll_onondaga_3), allrows = true)

animal = "Mollusca"
period = "Eifelian"

occs = pbdb_occurrences(
    base_name = animal,
    interval = period,
    show = "full",
    vocab = "pbdb",
    extids = true
)

size(occs)

combine(groupby(occs, :accepted_rank), nrow)

clean_taxonomy =
    row -> row.accepted_rank == "species" || row.accepted_rank == "subspecies"
occs_species = filter(clean_taxonomy, occs)

occs_species[:, r"._ma.*"]
```

```
println("Max MA measurements: ", nrow(dropmissing(occs_species, r".*max_ma.*"))))
println("Min MA measurements: ", nrow(dropmissing(occs_species, r".*min_ma.*"))))

occs_with_ages = dropmissing(occs_species,
                             [:max_ma, :min_ma])

modern_coords = nrow(dropmissing(occs_with_ages, [:lng, :lat]))
println("Records with modern coordinates: ", modern_coords)

occs_clean = dropmissing(occs_with_ages, [:lng, :lat])

species_counts = combine(groupby(occs_clean, :accepted_name), nrow)
genus_counts    = combine(groupby(occs_clean, :genus), nrow)
family_counts   = combine(groupby(occs_clean, :family), nrow)

println("Species: ", nrow(species_counts))
println("Genera:  ", nrow(genus_counts))
println("Families: ", nrow(family_counts))
```

From our outputs, there are a total of 308 species, 162 genera, and 66 families belonging to the Mollusca phylum during the Eifelian stage of the Devonian period, in which many of these species can be found within the Onondaga Formation.

start

Now that you’ve analyzed more of the phylum or class that was found to be the most represented in Onondaga Formation, you’ll use Navigator again to locate another fossil site representative of the Pleistocene Epoch of the Quaternary Period. Find a circle labeled “Chowan River, coral thicket” near Albemarle Sound, which is a bay near North Carolina. Just like what we’ve done in Kingston, write down the names of all phyla and classes in the area, then underline the phylum or class with the most occurrences. Use the same workflow established in Parts 1 and 2 to analyze that phylum or class with `pbdb_occurrences`, then answer the following question:

Question 4: Briefly describe how the ocean fauna has changed from the time of the Kingston fauna to the time of the Chowan River fauna. [Fronimos & Novack-Gottshall, 2018]

end

3 Part 3: Examining patterns through geological time (by creating diversity curves)

We will now be looking at sea floor fauna in the Paleozoic Era and in the modern-day continents. Back in the Paleozoic, the phylum *Brachiopoda* consisted of most of the fauna present, while in the modern-day world, *Mollusca* and *Bivalvia* fulfill most of the fauna present in the sea floor. For this part of the tutorial, we will be creating diversity curves for all three phyla to create a graph of diversity over geological time. `pbdb_occurrences` will be used to retrieve data for all three phyla, and the data will be cleaned and transformed in preparation for the next few steps. Using *Brachiopoda* as an example, the following code is:

```
using PaleobiologyDB, PaleobiologyDB.Taxonomy, DataFrames
using Makie, CairoMakie, GeoMakie, ColorSchemes

PaleobiologyDB.set_autocaching!(true);

occs_competition_raw_1 = pbdb_occurrences(
    base_name = "Brachiopoda",
    show = "full",
    vocab = "pbdb"
)

occs_competition_1 = dropmissing(occs_competition_raw_1, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])

occs_competition_1 = Taxonomy.drop_unqualified_taxa(occs_competition_1, "genus")
```

To calculate a midpoint age (Ma) for each fossil occurrence for *Brachiopoda*, we will use arithmetic operations, using a period mark (.) before each function (e.g., `.+`, `./`) to ensure that the operation is applied to each element in the column. The midpoint age is calculated by taking the average of the maximum and minimum midpoint age values for each fossil occurrence, as follows:

```
occs_competition_1.midpoint_age = (
    occs_competition_1.max_ma .+ occs_competition_1.min_ma
) ./ 2
```

```
interval_competition_1 = combine(
  groupby(occs_competition_1, :early_interval),
  :max_ma => maximum => :interval_max_ma,
  :min_ma => minimum => :interval_min_ma,
)
```

As shown above, `combine` and `groupby` functions were used to group the data per geological intervals. Now, we will calculate midpoint ages (Ma) for each interval, using the same workflow as before. Then, we will use another `combine` and `groupby` function combination to group the data based on genera belonging to *Brachiopoda*. This is shown as follows:

```
interval_competition_1.interval_mid_ma = (
  interval_competition_1.interval_max_ma .+ interval_competition_1.interval_min_ma
) ./ 2

interval_competition_1 = sort(interval_competition_1, :interval_mid_ma, rev=true)

genera_competition_1 = combine(
  groupby(occs_competition_1, :genus),
  :midpoint_age => maximum => :fad_ma,
  :midpoint_age => minimum => :lad_ma,
)
```

Now, we will create a diversity curve for all genera belonging to *Brachiopoda* using a `for...end` argument, which initiates flow control with index number `i` (referring to both the row and the y-axis). This serves to count one diversity number per interval and to collect them inside a vector, as follows:

```
diversity = Int[]

for interval_row in eachrow(interval_competition_1)
  n = count(
    g -> g.fad_ma >= interval_row.interval_mid_ma >= g.lad_ma,
    eachrow(genera_competition_1)
  )
  push!(diversity, n)
end

interval_competition_1.diversity = diversity
```

You'll need to repeat this workflow for both *Mollusca* and *Bivalvia* while ensuring you use different variable names for each phylum (for the sake of distinguishing between them). Shown below is an example of how you can use the workflow for *Mollusca*:

```

using PaleobiologyDB, PaleobiologyDB.Taxonomy, DataFrames
using Makie, CairoMakie, GeoMakie, ColorSchemes

PaleobiologyDB.set_autocaching!(true);

occs_competition_raw_2 = pbdb_occurrences(
    base_name = "Mollusca",
    show = "full",
    vocab = "pbdb"
)

occs_competition_2 = dropmissing(occs_competition_raw_2, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])

occs_competition_2 = Taxonomy.drop_unqualified_taxa(occs_competition_2, "genus")

occs_competition_2.midpoint_age = (
    occs_competition_2.max_ma .+ occs_competition_2.min_ma
) ./ 2

interval_competition_2 = combine(
    groupby(occs_competition_2, :early_interval),
    :max_ma => maximum => :interval_max_ma,
    :min_ma => minimum => :interval_min_ma,
)

interval_competition_2.interval_mid_ma = (
    interval_competition_2.interval_max_ma .+ interval_competition_2.interval_min_ma
) ./ 2

interval_competition_2 = sort(interval_competition_2, :interval_mid_ma, rev=true)

genera_competition_2 = combine(
    groupby(occs_competition_2, :genus),
    :midpoint_age => maximum => :fad_ma,
    :midpoint_age => minimum => :lad_ma,
)

```

```

diversity = Int[]

for interval_row in eachrow(interval_competition_2)
    n = count(
        g -> g.fad_ma >= interval_row.interval_mid_ma >= g.lad_ma,
        eachrow(genera_competition_2)
    )
    push!(diversity, n)
end

interval_competition_2.diversity = diversity

```

Once you obtain all three diversity loops, we'll need to create separate DataFrames for each diversity loop and intersecting intervals, which can be used to create our graph for range-through-diversity curves. For this section, we will be manually creating DataFrames with both `diversity` and `interval_mid_ma` columns for each `interval_competition` variable created for each phylum. You can use this code below:

```

timescale_1 = DataFrame(
    brachiopoda_diversity = interval_competition_1.diversity,
    interval_mid_ma_brach = interval_competition_1.interval_mid_ma
)

timescale_2 = DataFrame(
    mollusca_diversity = interval_competition_2.diversity,
    interval_mid_ma_mollus = interval_competition_2.interval_mid_ma
)

timescale_3 = DataFrame(
    bivalvia_diversity = interval_competition_3.diversity,
    interval_mid_ma_bival = interval_competition_3.interval_mid_ma
)

```

This will directly create three DataFrames for each phylum without needing to create complex functions. Now, you can create a diversity curve using “Makie” family packages (e.g., “Cairo-Makie” and “Makie”) to create a three-line graph with each phylum represented by different colors. The x-axis will represent the age (Ma), while the y-axis represents the number of genera present per intervals for each phylum. The code for this is as follows:

```

using Makie, CairoMakie, ColorSchemes

```

```

fig = Figure(size = (1750, 550), fontsize = 11.5)
ax1 = Axis(
    fig[1,1],
    xlabel = "Age (Ma)",
    ylabel = "Number of genera",
    title = "Genus diversity of fossils from Brachiopoda,
            Mollusca, and Bivalvia through time",
    xticks = (interval_competition_1.interval_mid_ma,
              interval_competition_1.early_interval),
    xticklabelrotation = /4,
    xticklabelsize = 4.5,
    xreversed = true,
)
ax2 = Axis(
    fig[1,1],
    xticks = (interval_competition_2.interval_mid_ma,
              interval_competition_2.early_interval),
    xticklabelrotation = /4,
    xticklabelsize = 4.5,
    xreversed = true,
)
ax3 = Axis(
    fig[1,1],
    xticks = (interval_competition_3.interval_mid_ma,
              interval_competition_3.early_interval),
    xticklabelrotation = /4,
    xticklabelsize = 4.5,
    xreversed = true,
)
hidedecorations!(ax2)
hidedecorations!(ax3)

lines!(ax1, interval_competition_1.interval_mid_ma,
        interval_competition_1.diversity, linewidth = 2, color = :orange)
scatter!(ax1, interval_competition_1.interval_mid_ma,
          interval_competition_1.diversity, markersize = 8, color = :orange, label = "Brachiopoda")

lines!(ax2, interval_competition_2.interval_mid_ma,
        interval_competition_2.diversity, linewidth = 2, color = :seagreen3)
scatter!(ax2, interval_competition_2.interval_mid_ma,
          interval_competition_2.diversity, markersize = 8, color = :seagreen3, label = "Mollusca")

```

```

lines!(ax3, interval_competition_3.interval_mid_ma,
        interval_competition_3.diversity, linewidth = 2, color = :royalblue)
scatter!(ax3, interval_competition_3.interval_mid_ma,
          interval_competition_3.diversity, markersize = 8, color = :royalblue, label = "Bivalvia")

axislegend(ax1, position = (0.01, 0.925), framevisible = true, labelsiz = 11)
axislegend(ax2, position = (0.01, 0.805), framevisible = true, labelsiz = 11)
axislegend(ax3, position = (0.01, 0.675), framevisible = true, labelsiz = 11)

fig

```

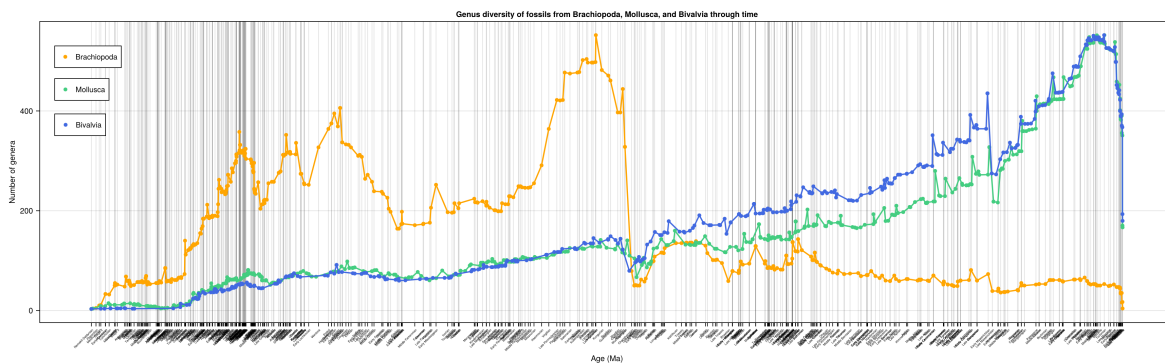


Figure 2: Three-line diversity curve for Brachiopoda, Mollusca, and Bivalvia over geological time

As you can tell from your chart, you'll have created three separate lines with distinguishable colors. The x-axis represents the age (Ma) of all fossil occurrences for our three phyla, and the y-axis represents the number of genera present per interval for each phylum. If you want, you can also add a legend to your graph to ensure that it is easier to understand what line represents which phylum.

Ensure that you save your map for reference, and then answer the following questions, which most of them stem from the original activity PDF. Questions 5a to 7 all stem from the original activity file (from Fronimos & Novack-Gottshall, 2018), while Question 8 is an original question that is based off on Question 7 (which stems from the original activity).

start

Question 5a: During what periods were the brachiopods more diverse than the bivalves? [Fronimos & Novack-Gottshall, 2018]

Question 5b: During what periods were the bivalves more diverse than the brachiopods? [Fronimos & Novack-Gottshall, 2018]

Question 5c: When, in geologic time, does the change from there being more brachiopods to more bivalves happen? What major event in Earth’s history was happening at that time? (You may use the Internet to further flesh out your answer.) [Fronimos & Novack-Gottshall, 2018]

Question 6: Many paleontologists have hypothesized that bivalves and brachiopods were competitors in the environment for food and space. Over time, the bivalves outcompeted the brachiopods and took over the environment. We can test this hypothesis using our data (and our graph from Julia). Consider the following: If it is true, we should be seeing a double-wedge pattern, where the diversity of brachiopods decreases whenever bivalve diversity increases. Now, let’s consider the Paleozoic by itself. When bivalve diversity goes up, does brachiopod diversity increase, decrease, or stay the same? Based on this result, do you think bivalves were gradually outcompeting brachiopods in the Paleozoic? [Fronimos & Novack-Gottshall, 2018]

Question 7: Resources like PaleobiologyDB (e.g., resources, Navigator) are relatively new tools in palentology and the research surrounding this field. We have only begun to scratch the surface of what can be done with “Big Data.” What use can you think of for this kind of dataset? Name at least one question you think could be answered or investigated using the PBDB. Explain the hypothesis that you would test and how you could use the PBDB to test it. [Fronimos & Novack-Gottshall, 2018]

Question 8: Now, what is one question that you think can be answered or investigated using Julia? Explain what hypothesis you would like to test, and how you could use Julia to test it. What packages? What functions and commands? What kind of high-quality visualizations would you like to create?

end

4 References (for Original Activity)

Fronimos, J., & Novack-Gottshall, P. (2018, May 2). Ships that pass in the night: Competition in the fossil record. Teach the Earth. <https://serc.carleton.edu/teachearth/activities/204123.html>